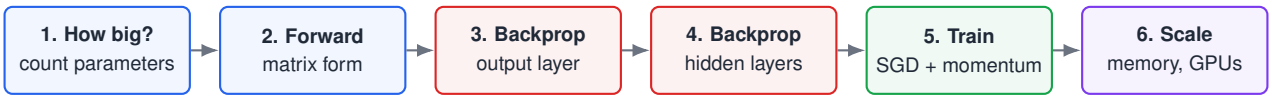
Feed-Forward Networks and Backpropagation

Explained simply · intuition first, then the maths, then the repo's NumPy code

Folder: `02 - Feed-Forward Networks and Backpropagation` — six notebooks, `01_...` to `06_...`

Where we are. The previous guide ended with a problem: one sigmoid neuron draws one straight line, but a *layer* of neurons can bend the boundary (e.g. solve XOR). This guide answers the obvious next questions:



Each numbered section below matches one notebook. Every number quoted was produced by that notebook's code.

Notation in one table (keep this page open!)

Symbol	Shape	Meaning in plain words
N	number	batch size: how many examples we process together
$X = A^{[0]}$	(N, d)	the input batch: **one row per example**, one column per feature
$W^{[l]}$	(n_{l-1}, n_l)	weights of layer l: column j = all incoming weights of neuron j
$b^{[l]}$	$(n_l,)$	biases of layer l: one per neuron
$Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]}$	(N, n_l)	"net" input of every neuron for every example (before activation)
$A^{[l]} = g(Z^{[l]})$	(N, n_l)	output (activation) of layer l
$\delta^{[l]} = \partial L / \partial Z^{[l]}$	(N, n_l)	**error signal**: how much the loss changes if $Z^{[l]}$ wiggles – the "blame"
η	number	learning rate

The handwritten notes write one column vector per example ($o = \sigma(Wx + b)$, W of shape $n_{\text{out}} \times n_{\text{in}}$). The code stacks examples as rows, so everything is transposed: $Z = AW + b$. Same maths, just turned on its side.

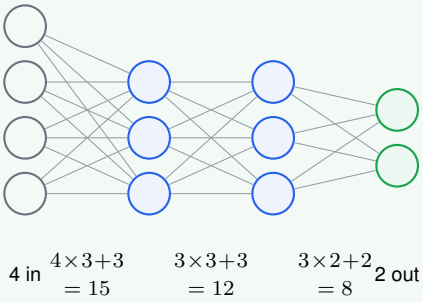
1 Architecture and Parameter Counting

notebook 01

A **fully-connected** (dense) layer connects *every* neuron of the previous layer to *every* neuron of this layer. Each connection is one weight; each neuron also has one bias. So:

$$\#\text{params of layer } l = \underbrace{n_{l-1} \times n_l}_{\text{weights (arrows)}} + \underbrace{n_l}_{\text{biases (one per neuron)}}$$

The network from the handwritten notes: 4 – 3 – 3 – 2



Count the arrows between each pair of layers, then add one bias per neuron:

$$\underbrace{12+3}_{15}+\underbrace{9+3}_{12}+\underbrace{6+2}_{8}=\mathbf{35}\ \text{parameters.}$$

Exactly the total written in the notes ("12+9+6 = 27 weights, + 8 biases = 35").

In the code: `count_params`

```python
def count_params(sizes):                          # sizes = [n0, n1, ..., nL]
    rows, total = [], 0
    for l, (n_in, n_out) in enumerate(zip(sizes[:-1], sizes[1:]), start=1):
        w, b = n_in * n_out, n_out                 # weights + biases of this layer
        rows.append(dict(layer=l, shape=(n_in, n_out), weights=w, biases=b, total=w + b))
        total += w + b
    return rows, total

rows, total = count_params([784, 128, 64, 10])    # an MNIST-sized classifier -> 109,386
```

`zip(sizes[:-1], sizes[1:])` pairs each layer with the next: $(784,128),(128,64),(64,10)$. The notebook then really allocates the arrays and checks that `W.size + b.size` adds up to the same number.

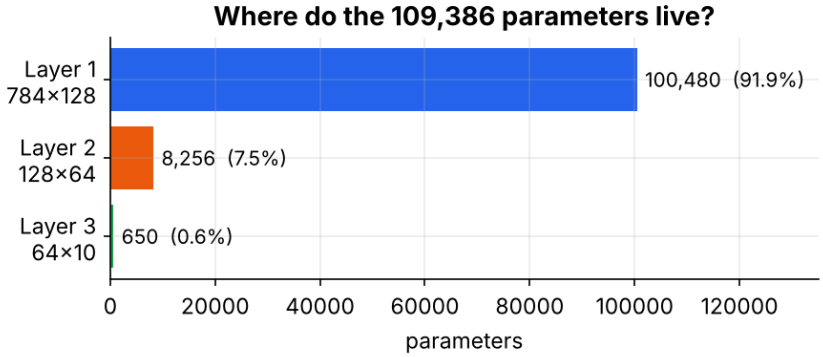


Figure 1: For a 28×28 image (784 pixels), the first layer holds 92% of all weights.

What the count tells you

- **Memory:** each parameter is a 4-byte float, so 109,386 params ≈ 0.44 MB.
- **Compute:** one forward pass does one multiply + one add per weight ⇒ about $2\times$(#weights) operations per example.
- **Width vs depth:** `784-512-10` has 407,050 params; `784-128-128-128-128-10` has only 151,306. Going *deeper* with narrow layers is cheap; making the first layer *wide* is expensive.

2 The Matrix-Form Forward Pass

notebook 02

2.1 One neuron = one dot product

Same as the perceptron: $z = \vec{w} \cdot \vec{x} + b$, then $a = g(z)$.

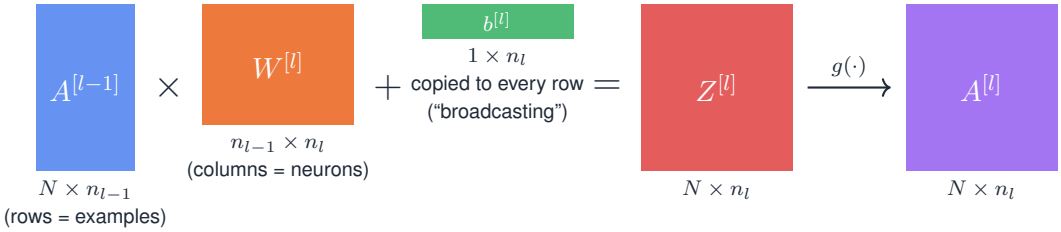
One neuron by hand (the notebook's numbers)

$\vec{x} = [1,\ 0.5,\ -1]$, $\vec{w} = [0.2,\ -0.4,\ 0.1]$, $b = 0.3$: $z = 1(0.2) + 0.5(-0.4) + (-1)(0.1) + 0.3 = 0.2 - 0.2 - 0.1 + 0.3 = \mathbf{0.2}$.

The loop (`z += w[i]*x[i]`) and the one-liner `x @ w + b` give the same 0.2.

2.2 A whole layer, a whole batch = one matrix multiply

Put each neuron's weights in a *column* of W, and each example in a *row* of A. Then entry (n, j) of AW is "example n dotted with neuron j's weights" – exactly that neuron's z for that example. All of them at once:



$$Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]}, \qquad A^{[l]} = g\big(Z^{[l]}\big), \qquad A^{[0]} = X$$

Shape trick: the inner sizes must match (n_{l-1}), and the outer sizes give the result ($N \times n_l$). Notice N never appears in W or b: the same weights are reused for every example, so you can feed a batch of any size.

Why bother with matrices?

Three nested loops (examples $\times$ neurons $\times$ inputs) become one call that NumPy hands to highly optimised code – and later to a GPU. In the notebook, a 32×256 by 256×64 layer took about 201 ms with Python loops and 0.145 ms as a matrix product: **over 1000$\times$ faster**, same numbers (checked with `np.allclose`).

2.3 Activation functions: the "bend" in each layer

Without g, layers collapse into one ($W_2(W_1x) = (W_2W_1)x$, as in the previous guide). Common choices:

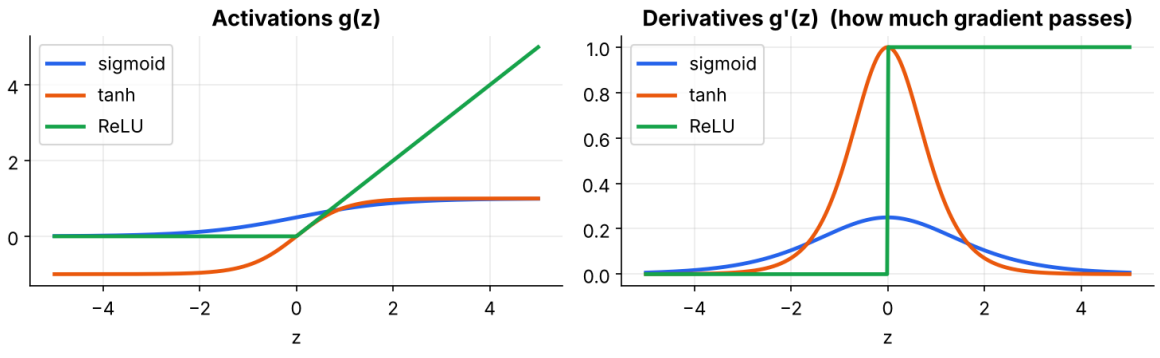


Figure 2: Left: the functions. Right: their slopes. The slope decides how much "blame" gets through during backprop (Section 4).

	Formula	Slope $g'(z)$	Good / bad
Sigmoid	$\frac{1}{1+e^{-z}}$	$a(1-a) \le 0.25$	output in $(0,1)$; slope small $\Rightarrow$ gradients shrink
tanh	$\tanh z$	$1-a^2 \le 1$	centred at 0; still flat at the ends
ReLU	$\max(0,z)$	1 if $z>0$, else 0	cheap, slope 1 keeps gradients alive; "dead" if $z<0$

2.4 Softmax: turning the last layer into probabilities

For K classes the last layer outputs K numbers ("logits"). Softmax makes them positive and sum to 1:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Softmax by hand

$z = [2,\ 1,\ 0.1]$: $e^z = [7.389,\ 2.718,\ 1.105]$, sum $= 11.212$, so $p = [0.659,\ 0.242,\ 0.099]$. Biggest logit $\rightarrow$ biggest probability, and they add to 1.

The stability trick. For $z = [1000, 1001, 1002]$, e^{1000} overflows to ∞ and the answer is `NaN`. But subtracting the same number from every logit doesn't change softmax (it cancels top and bottom). Subtract the max: $[-2, -1, 0] \Rightarrow p = [0.090,\ 0.245,\ 0.665]$. That is the line `z = z - z.max(axis=1, keepdims=True)` in the code.

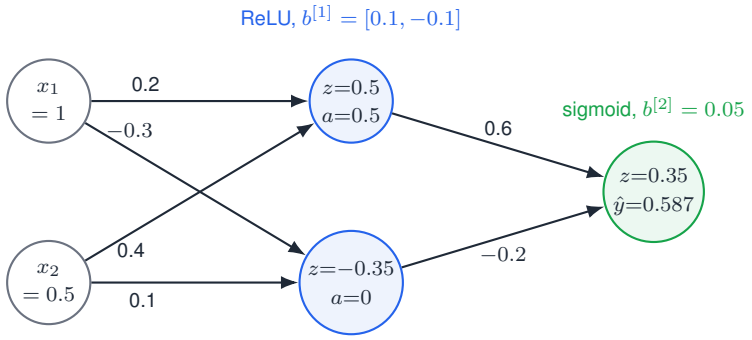
In the code: the full forward pass (keeps a "cache" for backprop)

```
def softmax(z):
    z = z - z.max(axis=1, keepdims=True)      # stability trick
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)   # each row sums to 1

def forward(X, params):
    A, L = X, len(params)
    cache = [(None, X)]                       # cache[l] = (Z[l], A[l])
    for l, (W, b) in enumerate(params, start=1):
        Z = A @ W + b                         # (N, n_in) @ (n_in, n_out) + (n_out,)
        A = softmax(Z) if l == L else relu(Z) # ReLU inside, softmax at the end
        cache.append((Z, A))                  # SAVE them: backprop needs Z and A
    return A, cache
# layer 1: A(32,784) @ W(784,128) + b(128,) -> Z(32,128) --ReLU-->    A(32,128)
# layer 3: A(32, 64) @ W( 64, 10) + b( 10,) -> Z(32, 10) --softmax--> A(32, 10)
```

2.5 A forward pass you can check with pen and paper (2 – 2 – 1 network)

This tiny network is used again in Section 4 for a full backprop step, so it's worth following.



$$z_1^{[1]} = 1(0.2) + 0.5(0.4) + 0.1 = 0.5 \qquad \Rightarrow\ a_1^{[1]} = \text{ReLU}(0.5) = 0.5$$
$$z_2^{[1]} = 1(-0.3) + 0.5(0.1) - 0.1 = -0.35 \qquad \Rightarrow\ a_2^{[1]} = \text{ReLU}(-0.35) = 0$$
$$z^{[2]} = 0.5(0.6) + 0(-0.2) + 0.05 = 0.35 \qquad \Rightarrow\ \hat{y} = \sigma(0.35) = 0.5866$$

3 Backpropagation: the Big Idea

Training needs, for *every* weight, the answer to one question: **"If I nudge this weight up a little, does the loss go up or down, and by how much?"** That is $\partial L/\partial w$. A network can have millions of weights; backprop gets *all* of these slopes in one backward sweep.

Intuition: passing the blame backwards

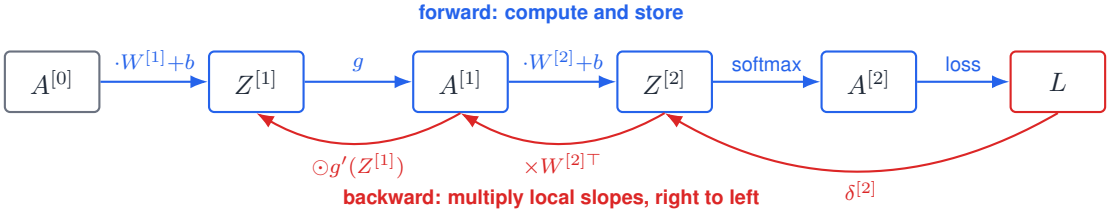
Think of a factory line: raw material → worker 1 → worker 2 → worker 3 → final product. The customer complains (the loss). The manager first blames worker 3, who directly made the product. Worker 3 then says "part of it was because of what worker 2 handed me", and passes some blame back – *in proportion to how much worker 2's part mattered*. And so on to worker 1. Each worker only needs to talk to their neighbours. The notes say the same: *"For backprop you need only local information"* and *"Only need info about next layer and previous layer."*

3.1 The only maths tool: the chain rule

If L depends on z, and z depends on w, then

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w}.$$

Everyday example. Petrol cost depends on km driven (Rs. 8/km), km depends on hours (60 km/h). So cost per hour = $8 \times 60 =$ Rs. 480/h. Rates multiply along a chain. Backprop is just this, applied layer by layer from the end towards the start.



3.2 The key quantity: the error signal δ

Define $\delta^{[l]} = \partial L / \partial Z^{[l]}$: "how much would the loss change if this neuron's net input moved?" Once we know δ for a layer, the weight gradients of that layer are easy (Section 3), and δ of the layer below is easy too (Section 4).

Sign conventions: notes vs code

The handwritten notes (following Mitchell's textbook) define $\delta_j = -\partial E / \partial net_j$ and *add*: $\Delta w_{ji} = +\eta\, \delta_j\, x_{ji}$. The code defines $\delta = +\partial L / \partial Z$ and *subtracts*: `W -= lr * dW`. Two minus signs moved around – the update is identical.

4 Backprop for the Output Layer

notebook 03

4.1 From δ to the weight gradient: start with one weight

Look at one weight w_{ij} connecting input i to output neuron j, one example. Since $z_j = \sum_i a_i w_{ij} + b_j$:

$$\frac{\partial L}{\partial w_{ij}} = \underbrace{\frac{\partial L}{\partial z_j}}_{\delta_j} \cdot \underbrace{\frac{\partial z_j}{\partial w_{ij}}}_{a_i} = a_i\, \delta_j \qquad \frac{\partial L}{\partial b_j} = \delta_j \cdot 1 = \delta_j.$$

In words: gradient of a weight = (what came *in* on that wire) $\times$ (blame at the neuron it goes *to*). A wire carrying 0 gets no blame – same as in the perceptron rule.

4.2 Now for all weights and all examples at once

Doing $a_i \delta_j$ for every (i, j) and adding over the N examples is exactly a matrix product. Let shapes guide you:

We want	Must have shape	Only way to build it	Code
$dW = \partial L / \partial W$	$(n_{\text{in}}, n_{\text{out}})$ like W	$A_{\text{prev}}^{\top}\,(n_{\text{in}} \times N)\ \cdot\ \delta\,(N \times n_{\text{out}})$	`A_prev.T @ delta`
db	$(n_{\text{out}},)$ like b	add δ over the N rows	`delta.sum(axis=0)`
dA_{prev}	(N, n_{in}) like A_{prev}	$\delta\,(N \times n_{\text{out}})\ \cdot\ W^{\top}\,(n_{\text{out}} \times n_{\text{in}})$	`delta @ W.T`

Tip: a gradient always has the same shape as the thing it is the gradient of. If you are unsure where the transpose goes, match the shapes – there is usually only one way that works.

4.3 The magic of softmax + cross-entropy: $\delta = P - Y$

For classification, the loss is **cross-entropy**: $L = -\sum_k y_k \log p_k$, with Y one-hot (a 1 at the true class). It just means "minus log of the probability you gave to the right answer": confident and right $\Rightarrow -\log(0.99) \approx 0.01$; confident and wrong $\Rightarrow -\log(0.01) \approx 4.6$.

Chain rule through softmax (the derivative of p_k w.r.t. z_j is $p_k(\mathbb{1}[k{=}j] - p_j)$):

$$\frac{\partial L}{\partial z_j} = \sum_k \Big(-\frac{y_k}{p_k}\Big)\, p_k\big(\mathbb{1}[k{=}j] - p_j\big) = -y_j + p_j \underbrace{\sum_k y_k}_{=1} = \boxed{p_j - y_j}$$

Everything cancels! Averaging over a batch adds $1/N$: $\delta^{[L]} = (P - Y)/N$.

What $P - Y$ means

3 classes, true class is #2. Network says $p = [0.7,\ 0.2,\ 0.1]$, so $y = [0, 1, 0]$ and

$$\delta = p - y = [\,+0.7,\ -0.8,\ +0.1\,].$$

Positive ⇒ "this logit is too high, push it down" (class 1 strongly, class 3 a little). Negative ⇒ "too low, push it up" (the true class). If the network had said $p = [0, 1, 0]$, then $\delta = 0$: nothing to fix.

In the code: output layer forward + backward

```
Z = A_prev @ W + b                # forward
P = softmax(Z)
loss = cross_entropy(P, Y)        # -sum(Y * log P) / N

delta   = (P - Y) / N             # dL/dZ           (N, K)
dW      = A_prev.T @ delta        # (d, N) @ (N, K) -> (d, K)  same shape as W
db      = delta.sum(axis=0)       # (K,)                       same shape as b
dA_prev = delta @ W.T             # (N, K) @ (K, d) -> (N, d)  blame for the layer
    below
```

4.4 Gradient checking: how to know your backprop is right

Measure the slope the "dumb" way – wiggle one weight up and down by a tiny ε and see how the loss changes:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \varepsilon) - L(\theta - \varepsilon)}{2\varepsilon}, \qquad \varepsilon = 10^{-6}.$$

This is slow (two forward passes per weight) but almost impossible to get wrong. Compare it with backprop's answer using the relative error $\frac{\|a-b\|}{\|a\|+\|b\|}$:

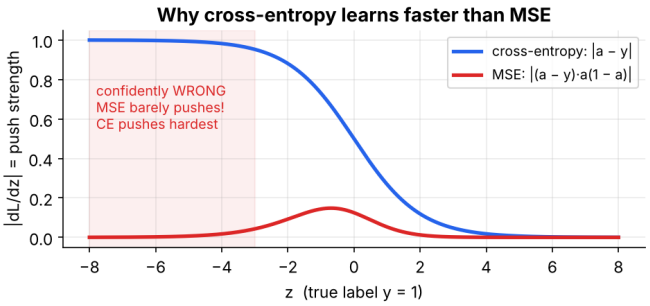
In the code: `numerical_grad`

```
def numerical_grad(f, x, eps=1e-6):
    g = np.zeros_like(x)
    it = np.nditer(x, flags=["multi_index"])
    for _ in it:                                  # visit every entry of x
        idx = it.multi_index; old = x[idx]
        x[idx] = old + eps; fp = f()              # loss with weight nudged up
        x[idx] = old - eps; fm = f()              # loss with weight nudged down
        x[idx] = old                              # put it back!
        g[idx] = (fp - fm) / (2 * eps)            # slope = rise / run
    return g
# notebook output:  dW 1.96e-10   db 7.97e-10   dA_prev 6.00e-10
```

Rule of thumb: below 10^{-6} = correct; above 10^{-3} = bug (forgot `/N`, wrong transpose, wrong g'...).

4.5 Why cross-entropy, not squared error, for classification?

Task	Output	Loss	Output error $\delta^{[L]}$
multi-class	softmax	cross-entropy	$(P - Y)/N$
binary	sigmoid	binary cross-entropy	$(A - Y)/N$
regression	identity	$\frac{1}{2}$ squared error	$(A - Y)/N$
binary	sigmoid	squared error	$(A - Y)\,A(1 - A)/N$ ← slow!



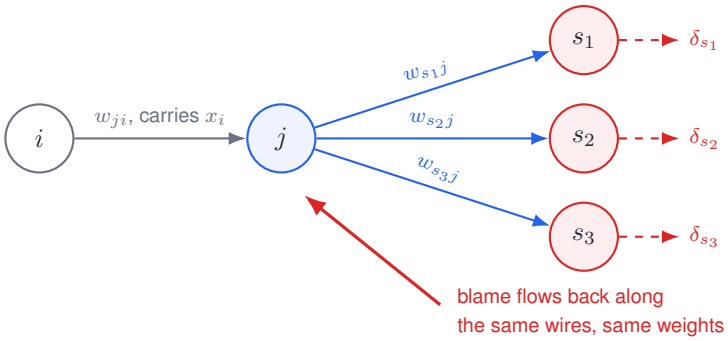
With squared error, an extra factor $a(1 - a)$ (the sigmoid's slope) sneaks in. When the neuron is **confidently wrong** (true $y = 1$ but z very negative, so $a \approx 0$), that factor is ≈ 0 and learning nearly stops – exactly when it should be fastest! Cross-entropy's $\log$ cancels the factor, so the push is $|a - y| \approx 1$: big mistakes get big corrections.

5 Backprop Through Hidden Layers

notebook 04

5.1 How a hidden neuron gets its blame

A hidden neuron j doesn't touch the loss directly. It only affects the loss *through* the neurons s it feeds into (its "downstream" neurons). So its blame is the sum of their blames, each weighted by the wire w_{sj} that connects them, then scaled by how sensitive neuron j itself is:



$$\delta_j = \underbrace{g'(z_j)}_{\text{my sensitivity}} \cdot \sum_{s \in \text{downstream}(j)} \underbrace{\delta_s}_{\text{their blame}} \underbrace{w_{sj}}_{\text{how much I mattered to them}} \qquad \Delta w_{ji} = -\eta\, \delta_j\, x_i$$

This is exactly the boxed formula in the handwritten notes, $\Delta w_{ji} = \eta\, x_i\, o_j(1 - o_j) \sum_s \delta_s w_{sj}$, with $g' = o_j(1 - o_j)$ for a sigmoid (and the notes' sign convention).

5.2 Matrix form: two lines per layer

Stack all neurons and examples, and the sum over s becomes a product with $W^\top$:

$$\delta^{[l]} = \left(\delta^{[l+1]}\,(W^{[l+1]})^\top\right) \odot g'(Z^{[l]}) \qquad \frac{\partial L}{\partial W^{[l]}} = (A^{[l-1]})^\top \delta^{[l]}, \qquad \frac{\partial L}{\partial b^{[l]}} = \sum_{\text{rows}} \delta^{[l]}$$

$\odot$ means element-by-element multiply. Read it as: **"take the blame from the layer above, send it back through the weights, then gate it by this layer's slope."**

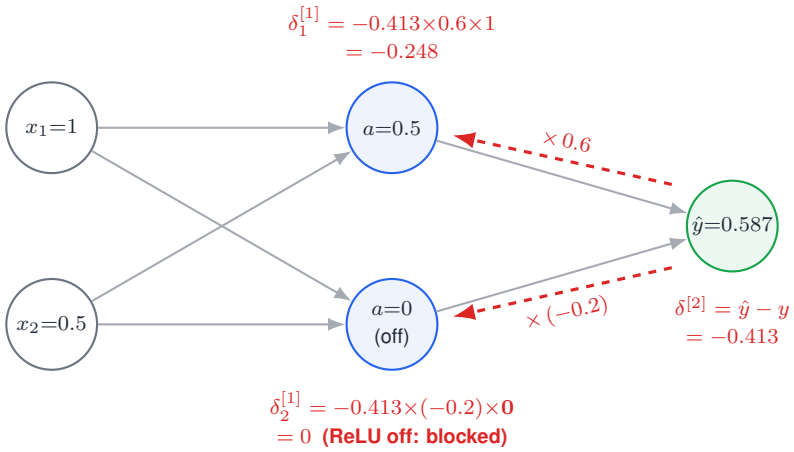
In the code: `backward` – the whole algorithm

```
1  def backward(Y, params, cache, act="relu"):
2      _, gprime = ACT[act]                          # derivative of the activation
3      N, L = Y.shape[0], len(params)
4      grads = [None] * L
5      delta = (cache[-1][1] - Y) / N                # output layer: (P - Y) / N
6      for l in range(L, 0, -1):                     # l = L, L-1, ..., 1  (backwards!)
7          A_prev = cache[l - 1][1]                  # stored in the forward pass
8          W, _ = params[l - 1]
9          grads[l - 1] = (A_prev.T @ delta,         # dW
10                         delta.sum(axis=0))        # db
11         if l > 1:
12             Z_prev = cache[l - 1][0]              # stored in the forward pass
13             delta = (delta @ W.T) * gprime(Z_prev)    # send back, gate by g'(Z)
14     return grads
15 # gradient check on a 3-hidden-layer tanh network: every dW, db has rel. error ~1e-9
```

Lines 9–10 are "my gradient = what came in $\times$ my blame". Line 13 is "pass the blame down". That's all of backprop. Note that it needs every Z and A from the forward pass – that is why we kept the `cache` (and why training needs more memory than inference, Section 6).

5.3 Worked example: one full backprop step by hand

The 2–2–1 network from Section 2.5. Target $y = 1$, sigmoid output, binary cross-entropy loss $L = -\ln \hat{y} = -\ln 0.5866 = 0.5334$.



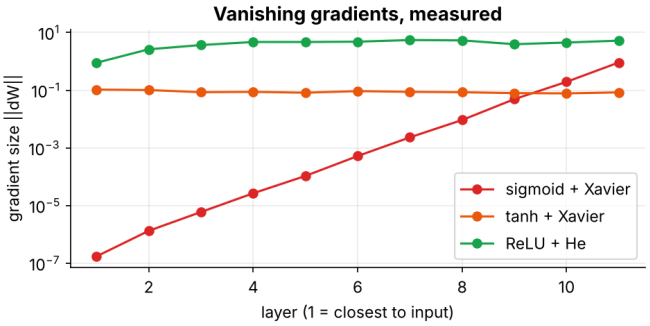
Step	Computation	Result
1. Output blame	$\delta^{[2]} = \hat{y} - y = 0.5866 - 1$	-0.4134
2. Output weights	$dW^{[2]} = a^{[1]} \cdot \delta^{[2]} = [0.5,\ 0] \times (-0.4134);\ db^{[2]} = \delta^{[2]}$	$[-0.2067,\ 0];\ -0.4134$
3. Send blame back	$\delta^{[2]} \cdot W^{[2]\top} = -0.4134 \times [0.6,\ -0.2]$	$[-0.2480,\ +0.0827]$
4. Gate by ReLU slope	$g'(z^{[1]}) = g'([0.5,\ -0.35]) = [1,\ 0]$	$\delta^{[1]} = [-0.2480,\ 0]$
5. Hidden weights	$dW^{[1]} = x^\top \delta^{[1]};\ db^{[1]} = \delta^{[1]}$	$\begin{bmatrix} -0.2480 & 0 \\ -0.1240 & 0 \end{bmatrix};\ [-0.2480,\ 0]$
6. Update ($\eta = 0.5$)	$\theta \leftarrow \theta - 0.5\, d\theta$, e.g. $w_1^{[2]}$: $0.6 \to 0.7033$	$w_{11}^{[1]}$: $0.2 \to 0.3240$, $b^{[2]}$: $0.05 \to 0.2567$
7. Forward again	same input, new weights	$\hat{y}$: $0.5866 \to 0.6910$; loss: $0.5334 \to 0.3697$

What just happened

The answer was too low (0.587 vs 1), so $\delta^{[2]} < 0$ and every weight on the active path was *increased* (subtracting a negative gradient). One step cut the loss by 31%.
The second hidden neuron was "off" ($z = -0.35 < 0$, ReLU slope 0), so **none of its weights changed** – the blame was blocked. If a ReLU neuron is off for every example, it never learns again: the "dying ReLU" problem in miniature.

5.4 Vanishing gradients: why deep sigmoid networks struggle

Every backward step multiplies the blame by $g'(z)$. For a sigmoid $g' \leq 0.25$. After 10 layers: $0.25^{10} \approx 10^{-6}$. The layers near the input get almost no signal and barely learn.



Measured in the notebook on a network with 10 hidden layers. Gradient size at **layer 1**:

- sigmoid: 1.7×10^{-7} (vs 0.92 at the last layer!)
- tanh: 0.105
- ReLU + He init: 0.90

ReLU's slope is exactly 1 on the active side, so the blame passes through undamaged. **He initialisation** (random weights with variance $2/n_{\text{in}}$) keeps the signal at a stable size from layer to layer.

Backprop in one sentence

One forward sweep (store every Z, A) + one backward sweep (two matrix products per layer: one for dW, one for the next δ). So backward costs about $2\times$ the forward pass, and you get *every* gradient. This is "reverse-mode automatic differentiation" – what PyTorch's `loss.backward()` does for you.

6 Training a Small Network

notebook 05

6.1 The training loop



Mini-batch SGD is the middle ground from the previous guide: not one example (too noisy), not all of them (too slow), but e.g. 32 at a time. Shuffling each epoch stops the network memorising the order.

6.2 Momentum: a heavy ball instead of a nervous walker

$$v \leftarrow \mu\, v - \eta\, g, \qquad \theta \leftarrow \theta + v \qquad (\mu \approx 0.9)$$

Intuition

Plain SGD steps exactly where the current (noisy) gradient points. With momentum, v is a running "velocity" that remembers the recent directions. Like a heavy ball rolling down a valley, it ignores small side-to-side jiggles and speeds up along directions that stay consistent – which fixes the slow "narrow valley" crawl we saw with the Delta rule.

In the code: the `MLP` class (`step` and `fit`)

```
def step(self, gW, gb, lr, momentum=0.0):
    for l in range(len(self.W)):
        self.vW[l] = momentum * self.vW[l] - lr * gW[l]    # update velocity
        self.vb[l] = momentum * self.vb[l] - lr * gb[l]
        self.W[l] += self.vW[l]                            # move by velocity
        self.b[l] += self.vb[l]

def fit(self, X, Y, Xte, Yte, epochs=100, batch=32, lr=0.1, momentum=0.0, seed=0):
    r = np.random.default_rng(seed)
    for ep in range(epochs):
        idx = r.permutation(len(X))                        # 1. shuffle
        for s in range(0, len(X), batch):
            j = idx[s:s + batch]                           #    mini-batch
            self.forward(X[j])                             # 2-3. forward
            gW, gb = self.backward(Y[j])                   # 4. backward
            self.step(gW, gb, lr, momentum)                # 5. update
```

With `momentum=0` this is plain SGD: $v = -\eta g$, so $\theta \leftarrow \theta - \eta g$. The `backward` here is the same algorithm as Section 4, with the ReLU slope written as `(self.Z[l] > 0)`.

6.3 Results

XOR (impossible for one neuron): a 2–8–2 network reaches predictions $[0, 1, 1, 0]$ with loss 1.8×10^{-4}.
3-arm spiral (much harder, very curved boundary): a 2–64–64–3 ReLU network, $\eta = 0.05$, momentum 0.9, 150 epochs:

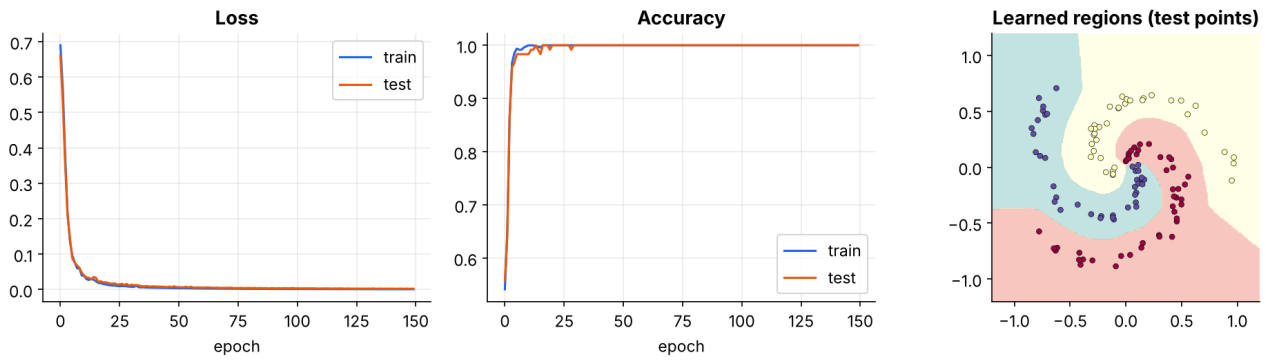
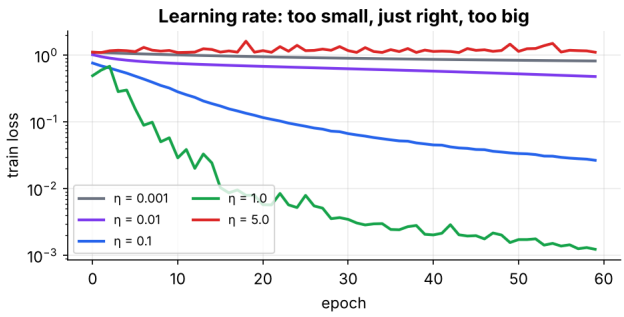


Figure 3: 100% train and 100% test accuracy. Right: the curved regions the network carved out – impossible with straight lines.

6.4 The learning rate is the most important knob



Plain SGD, 60 epochs, same network:

η	test acc.	verdict
0.001	52%	far too slow
0.01	85%	slow
0.1	99%	good
1.0	100%	good
5.0	33%	overshoots (= guessing)

Recipe: try η on a log scale (0.001, 0.01, 0.1, 1), watch the loss curve, keep the biggest one that still goes down smoothly.

7 Memory Footprint and Multi-GPU Training

notebook 06

7.1 What must sit in memory? (the notes' small example)

The handwritten notes use a 4–5–3–2 network: $(20{+}5) + (15{+}3) + (6{+}2) = \mathbf{51}$ parameters.

- **Inference (forward only):** we only need the current layer's input and output at any moment, then throw them away. Peak ≈ 51 params + 4 inputs + 5 hidden values.
- **Training (forward + backward):** backprop needs every layer's activations later, so we must *keep all of them*: 51 + 4 + 5 + 3 + 2 numbers (plus the gradients).

That is the core reason training needs much more memory than inference.

7.2 The 16-bytes-per-parameter rule

Training with the Adam optimiser in 32-bit floats keeps, **for every parameter**:

$$\underbrace{4}_{\text{weight}} + \underbrace{4}_{\text{gradient}} + \underbrace{8}_{\text{Adam's 2 running averages}} = \mathbf{16}\ \text{bytes} \quad + \ \text{activations (grow with batch size).}$$

Checked against real arrays in the notebook: `784-4096-4096-10` **MLP, batch 256**

20,037,642 parameters ⇒ weights 80.15 MB, gradients 80.15 MB, Adam states 160.30 MB, activations 17.60 MB. **Total 338 MB** – the estimate matched the real `nbytes` exactly.

At LLM scale (activations ignored): 1B params → 16 GB, 7B → 112 GB, 70B → 1,120 GB. A single 80 GB GPU can't even hold a 7B model's training state.

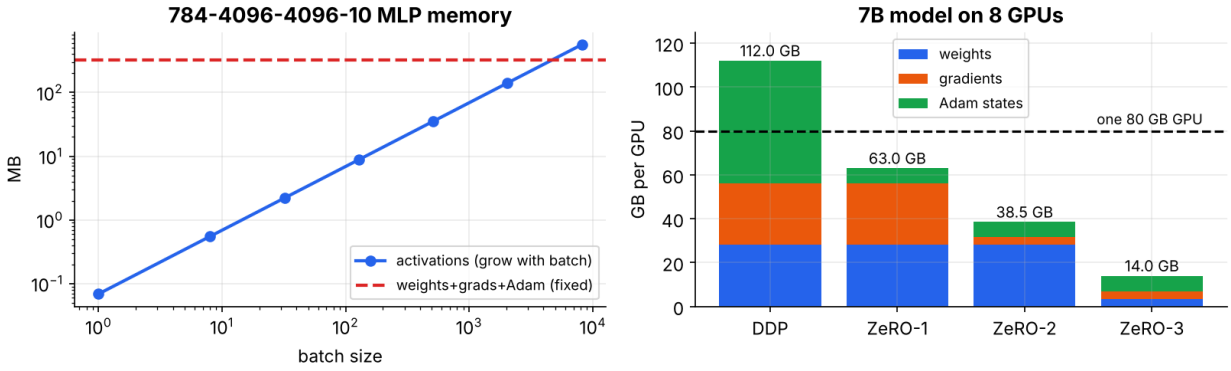


Figure 4: Left: parameters cost a fixed amount; activations grow with batch size. Right: per-GPU memory for a 7B model on 8 GPUs as more state is split up (ZeRO stages, below).

7.3 Using several GPUs

1) Data parallel – "photocopy the model, split the data". Every GPU holds a full copy of the model and processes a different slice of the batch. Then the GPUs **average** their gradients (an "all-reduce") and all apply the same update. This gives *exactly* the single-GPU answer because the loss is a mean:

$$g_{\text{full}} = \frac{1}{N}\sum_{n=1}^{N} g_n = \sum_{k=1}^{K} \frac{N_k}{N}\, g_k \qquad (g_k = \text{average gradient on GPU } k\text{'s } N_k \text{ examples}).$$

In the code: data parallelism, simulated

```
shards  = np.array_split(np.arange(N), K)        # each "GPU" gets N/K samples
# every GPU runs its own forward + backward on its slice:
g_local = [flat(grads_for(params, X[s], Y[s])) for s in shards]
# all-reduce = weighted average of the K gradients:
g_avg   = sum(len(s) / N * g for s, g in zip(shards, g_local))
# same as one GPU on the whole batch (max difference ~5.6e-17, just rounding):
assert np.allclose(flat(grads_for(params, X, Y)), g_avg)
```

The weights `len(s)/N` matter if the shards are unequal; a plain average would be slightly wrong.

In a **ring all-reduce** the GPUs sit in a circle and pass chunks to their neighbour. Each GPU sends only $\frac{2(K-1)}{K}$ times the gradient size – almost the same whether you have 4 GPUs or 400.

2) ZeRO / FSDP – "don't keep identical copies". Data parallel stores the same weights, gradients and Adam states on every GPU. ZeRO splits them: stage 1 splits Adam states, stage 2 also gradients, stage 3 also the weights. For 7B params on 8 GPUs: 112 → 63 → 38.5 → 14 GB per GPU.

3) Pipeline parallel – "split by layers" (the notes: *"Divide by layers"*). GPU 1 holds the first layers, GPU 2 the next, and they pass activations along. Backprop still works because each layer only needs local information from its neighbour. Forward F_i must come before backward B_i for each micro-batch.

4) Tensor parallel – "split one big matrix" (the notes: *"Weight matrix too big… split weight matrices"*). When even one layer doesn't fit, its weight matrix is cut into column blocks on different GPUs.

Technique	What is split	What is sent between GPUs	Problem it solves
Data parallel	the batch	gradients (all-reduce)	speed (throughput)
ZeRO / FSDP	optimizer states, grads, weights	pieces gathered when needed	memory per GPU
Pipeline parallel	layers	activations between stages	model too big for one GPU
Tensor parallel	each matrix multiply	partial results	a single layer too big

8 When Things Go Wrong, and a Cheat Sheet

Symptom	Likely cause	What to do
Loss becomes `NaN`	η too big, $\log(0)$, unstable softmax	lower η; subtract row max; add `1e-12` inside log
Loss stuck at $\ln K$ (e.g. 1.0986 for 3 classes)	network outputs uniform guesses: bad init, bad η	He init; print $\|dW\|$ per layer; shuffle data
Gradient check error $> 10^{-3}$	wrong transpose, missing `/N`, wrong g'	check layer by layer; use tanh for the check (ReLU has a kink)
Train acc. high, test acc. low	over-fitting	more data, smaller net, weight decay, early stopping
GPU out of memory	batch too big, optimizer states, activations	estimate 16 B/param + activations; smaller batch; ZeRO

Cheat sheet

Concept	Formula	Code
Params of a layer	$n_{\text{in}}\, n_{\text{out}} + n_{\text{out}}$	`n_in*n_out + n_out`
Forward	$Z = AW + b, \quad A = g(Z)$	`Z = A @ W + b`
Output error (softmax+CE)	$\delta^{[L]} = (P - Y)/N$	`delta = (P - Y) / N`
Weight / bias gradient	$dW = A_{\text{prev}}^{\top}\delta, \quad db = \sum_{\text{rows}} \delta$	`A_prev.T @ delta`
Hidden error	$\delta^{[l]} = (\delta^{[l+1]} W^{[l+1]\top}) \odot g'(Z^{[l]})$	`(delta @ W.T) * gprime(Z)`
SGD + momentum	$v \leftarrow \mu v - \eta g, \quad \theta \leftarrow \theta + v$	`v = m*v - lr*g; W += v`
Training memory (fp32, Adam)	16 B/param + activations	ZeRO-3: $16P/K$ per GPU

Check yourself

1. How many parameters does a `3-4-1` network have?
2. W has shape $(64, 10)$ and the batch is 32. What shapes are δ and dW?

3. Softmax gives $p = [0.1, 0.6, 0.3]$, true class is #3. What is δ (one example)?
4. Why did the second hidden neuron in the worked example get zero gradient?
5. Why does training a model need more memory than running it?

Answers. 1) $(3 \cdot 4 + 4) + (4 \cdot 1 + 1) = 21$. 2) δ: $(32, 10)$; dW: $(64, 10)$, same as W. 3) $[0.1,\ 0.6,\ -0.7]$. 4) Its z was negative, so the ReLU slope was 0 and blocked the blame. 5) Backprop needs every stored activation, plus gradients and optimizer states for every parameter.